

JORNADAS DATA SCIENCE CON IA

TU PRIMER DÍA COMO DATA SCIENTIST

APUNTES DÍA 2

[ACCEDE AL REPLAY DE LA SESIÓN 2 AQUÍ](#)

BIG school

DEL NEGOCIO, PASANDO POR EL DATO, A LA DECISIÓN

INTRODUCCIÓN

¡Bienvenido a los apuntes del Día 2 de las Jornadas Data Science con IA!

En el primer día sentamos las bases, y ayer por fin fuimos al grano: salimos de la teoría del laboratorio y nos metimos de lleno en una empresa real.

Como viste en el directo, la sesión giró en torno a un escenario práctico.

Imagina la situación que te planteé: te acaban de fichar como el primer Data Scientist de *StyleData*, un retailer de moda fast-fashion con tiendas en 6 países.

Los datos de la empresa llevan ahí meses, cogiendo polvo.

Tu misión era clara: **convertir esos números aburridos en decisiones que generen negocio.**

Cómo te he prometido, te he preparado este documento para que puedas masticar y repasar todo lo que hicimos en la demo a tu propio ritmo, sin agobios.

Aquí tienes tu hoja de ruta para ir del negocio, pasando por el dato, y directo a la decisión.

Lo que te llevas de estos apuntes:

- La mentalidad correcta que necesitas tener antes de abrir un dataset o picar una sola línea de código.
- Cómo funcionan las bases de datos relacionales y por qué el SQL sigue siendo el rey indiscutible de los datos.
- Cómo conectar una IA a tu base de datos usando Claude Code para analizar la información en lenguaje natural (como si chatearas con un compañero).
- El paso a paso metodológico para no perderte y ejecutar proyectos de Data Science de principio a fin.
- Proyecto 1 (Forecast de demanda): cómo predecir exactamente cuánto tienes que producir antes de que empiece la temporada.
- Proyecto 2 (RAG para contratos): cómo hacer que la IA se lea cientos de PDFs infumables y te responda al instante.
- Las 4 habilidades que de verdad separan a los profesionales que se estancan de los que escalan en su carrera.

1. TU BIENVENIDA A STYLEDATA — TU PRIMER DÍA COMO DATA SCIENTIST

Para aprender Data Science de verdad, la teoría de los libros no sirve de mucho si no la aplicas.

Necesitas un problema real, con datos reales y decisiones de negocio reales.

Por eso ayer te presenté a *StyleData*, un retailer de moda fast-fashion nacido en España y con tiendas en 6 mercados europeos (España, Francia, Italia, Alemania, Portugal y Reino Unido).

Imagina la situación:

acabas de aterrizar y eres el primer Data Scientist que contrata la empresa.

Tienen datos, claro. Tienen el registro de ventas, el inventario, las devoluciones y cientos de contratos con proveedores en PDF, todo metido en un data warehouse desde 2024. Pero esos datos llevan ahí meses, **cogiendo polvo**.

Nadie los ha usado todavía para tomar una sola decisión. El equipo de compras sigue pidiendo a ojo, el sobrestock se lo comen con rebajas de emergencia y nadie se lee los contratos porque buscar algo ahí es un infierno.

Quizás ahora que sabes el poder de esos datos, te preguntes cómo ha sobrevivido la empresa sin organizar, analizar sus datos para hacer crecer sus ventas.

Pero también es una oportunidad de demostrar tu visión estratégica de negocio, para deslumbrar al resto del equipo.

LO QUE HAY

- Datos de ventas por tienda, mercado y producto
- Inventario y devoluciones registradas
- Contratos con todos los proveedores en PDF
- Todo ingestado y listo en el data warehouse

LO QUE FALTA

- Ninguna decisión se ha tomado con datos hasta hoy
- No hay modelos de previsión de demanda
- No hay acceso rápido a las condiciones de contratos
- No hay cultura de datos en los equipos

Ese es tu punto de partida. Y tu misión no es hacer código bonito, tu misión es convertir esos datos en decisiones

2. ANTES DE TOCAR UN DATO — LA MENTALIDAD CORRECTA

"El error más caro de un Data Scientist no es un modelo que falla. Es resolver el problema equivocado."

Mira, te soy sincero. Uno de los errores más comunes cuando alguien empieza en esto es llegar con la solución técnica por delante, antes siquiera de haber entendido **cuál es el problema**.

Antes de abrir Jupyter Notebook y mucho antes de picar una sola línea de SQL, tienes que hacer algo mucho más importante: **escuchar al negocio**.

Y para eso, hace falta ser curioso. Te explico a que me refiero:

2.1 La curiosidad como herramienta profesional

La curiosidad aquí no es ser "cotilla", es una herramienta de trabajo que tienes que cultivar. Si no eres curioso, vas a dejar de hacer las preguntas incómodas, te tragarás los datos tal cual llegan y montarás modelos sobre suposiciones que nadie ha comprobado.

Y ojo, no hablo de curiosidad técnica ("¿qué pasa si cambio este hiperparámetro?").

Hablo de curiosidad de negocio.

Ayer lo vimos claro con un ejemplo en *StyleData*: las ventas de chaquetas en Francia caían en abril.

Un perfil sin curiosidad simplemente coge ese patrón y lo mete en el modelo. Un buen Data Scientist pregunta:

"Oye, ¿por qué caen en abril y no en marzo? ¿Es por el clima? ¿O es que Compras dejó de reponer stock a tiempo?".

Esa simple pregunta te puede revelar que el problema no es que la gente no quiera comprar, sino que reponemos tarde. **Y eso, un modelo predictivo nunca te lo va a solucionar solo.**

2.2 Escuchar de verdad — El arte de la entrevista con el negocio

Ya somos curiosos. Ya nos hemos preguntado por el problema. Pero otra cosa que hace a un buen Data Scientist, a uno que destaca, es el escuchar a los que YA conocen el negocio desde dentro: ¿qué les duele?

Oír no es escuchar. No tienes que convertirte en una grabadora. **Escuchar es entender qué hay detrás de la queja** de un director u otro *stakeholder*.

LAS REGLAS DE LA ESCUCHA ACTIVA

Cuando hables con ellos, pregúntales por el impacto en euros, no en procesos. Si alguien se queja del sobrestock, pregúntale: *"¿Cuánto margen calculas que perdemos en rebajas por este problema cada temporada?"*.

Así conviertes una queja de pasillo en un problema de negocio con un impacto económico brutal. Eso es lo que te ayuda a priorizar

La pregunta estrella que lo cambia todo es esta:

"¿Qué decisiones tomáis hoy sin datos que os gustaría tomar con datos?"

Haz esa pregunta y tendrás trabajo (del que aporta valor) para los próximos 6 meses. Esta es la pregunta que te dará el **roadmap** a seguir, ayudando al negocio a definir sus prioridades.

Toda respuesta lleva detrás un proyecto de datos esperando.

Ayer vimos las quejas reales que nos pasaron en *StyleData* y cómo traducirlas a proyectos:

- **Director de Compras:** *"Pedimos a ojo y siempre nos pasamos o nos quedamos cortos"*. → Proyecto: Modelo de forecast de demanda.
- **Responsable Comercial:** *"Perdemos muchísimo margen en rebajas"*. → Proyecto: Predicción de ventas por tienda para ajustar la producción.
- **Equipo de Compras:** *"Perdemos horas buscando condiciones en contratos en PDF"*. → Proyecto: Sistema RAG de IA para consultar contratos chateando

2.3 Tu valor está en las decisiones, no en los modelos

Grábate esto a fuego: un modelo de *machine learning*, por sí solo, no vale absolutamente nada ¿Qué significa esto?

✗ Que si haces un modelo de predicción con un 95% de precisión, pero Compras **no lo entiende** y sigue pidiendo a ojo, **tu impacto económico ha sido de cero euros**.

✓ Si haces uno con un 80% de precisión pero **el equipo lo usa** y reducimos el sobrestock un 25%, acabas de ahorrarle cientos de miles de euros a la empresa.

$$\text{VALOR} = \text{CALIDAD DEL MODELO} \times \text{ADOPCIÓN} \times \text{IMPACTO DE LA DECISIÓN}$$

Tu trabajo no termina cuando el código compila. Termina cuando la **decisión** de la empresa **mejora**.

2.4 Los 5 errores de mentalidad que se cargan tu carrera

Yo mismo he visto a perfiles muy técnicos fracasar por caer en estas trampas:

El pozo técnico:

Perder semanas intentando rascar un 2% más de precisión a un modelo cuando el negocio necesitaba una respuesta (aunque fuera imperfecta) hace dos semanas. **La perfección técnica no siempre compensa.**

Resolver el problema equivocado:

Ponerte a predecir ventas por categoría cuando lo que el negocio necesitaba saber era cuándo lanzar el pedido al proveedor (que depende de otras cosas como el lead time). Si no haces las **preguntas correctas el día 1**, tiras semanas de trabajo a la basura.

La torre de marfil:

Consiste en encerrarte a trabajar durante semanas en **aislamiento total, sin enseñar avances intermedios al negocio**, para luego aparecer con un análisis enorme que resulta que **nadie había pedido** exactamente así. Mientras tú trabajas, las prioridades y el contexto cambian. Para evitarlo, debes mostrar trabajo parcial pronto y con frecuencia: "Aquí tengo el primer análisis, ¿esto responde a lo que necesitáis o voy en la dirección equivocada?"

La caja negra:

Presentar un número ("hay que pedir 1.340 abrigos") sin explicar de dónde sale. Si el negocio no entiende tu modelo, no confiará en él y no lo usará. **La explicabilidad es innegociable.**

Medir lo fácil:

Al director comercial le da igual si tu modelo tiene un "AUC de 0.87". A él háblale en euros, en margen recuperado o en horas ahorradas. **Traduce siempre tu métrica técnica al idioma del dinero.**

Puede dar más miedo de lo que realmente representa. En realidad solo se trata de ser metódico y, cómo ya dijimos, curioso.

2.5 Entiende a quién impactas (No hablas solo con IT)

El Data Scientist que solo se relaciona con el equipo técnico se está perdiendo la mitad de su trabajo.

En una empresa como *StyleData*, vas a impactar a **Comercial** (les importan los márgenes y las ventas), a **Compras** (les preocupan los tiempos de entrega y el stock) y a **Marketing** (miran el ROI y la conversión).

DEPARTAMENTO	SU IDIOMA	SU FOCO (Y qué les duele)
 COMERCIAL	Hablan de: márgenes, conversión y categorías estrella.	Les importa qué se vende, dónde y cuándo. Su mayor dolor: Que el forecast (la predicción) falle y se vean obligados a hacer rebajas que destrozan su margen.
 COMPRAS	Hablan de: lead times (tiempos de entrega), mínimos de pedido, sobrestock y roturas de stock.	Toman decisiones hoy para cosas que llegarán con 3-4 meses de antelación. Necesitan: Certeza absoluta antes de comprometer el presupuesto de la empresa.
 MARKETING	Hablan de: ROI de campañas, segmentos, churn (abandono de clientes) y conversión.	Necesitan: Que los datos les digan a quién dirigirse exactamente y qué funciona mejor en cada canal para no desperdiciar el presupuesto.

Cada área tiene métricas de éxito distintas y presiones del día a día muy diferentes. Piénsalo de esta forma: un análisis perfectamente presentado para el director de compras puede resultarle incomprensible al de marketing si usas el mismo vocabulario.

Aprender a comunicarte en el idioma de cada interlocutor no es un "extra", es una parte fundamental de tu trabajo.

2.6 Cómo priorizar — La matriz impacto vs. esfuerzo

Tu **tiempo** es, literalmente, el recurso más caro que tiene la empresa ahora mismo.

Por eso, el primer mes no puedes intentar apagar todos los fuegos a la vez.

La trampa más habitual en la que caen los juniors es coger un proyecto simplemente porque "técnicamente es un reto muy chulo", aunque luego su impacto real en la facturación sea bajísimo. Huye de ahí.

Tú estás en la empresa para mover la aguja. **Siempre debes cruzar dos variables:**

¿Cuánto esfuerzo técnico me va a llevar esto?

vs.

¿Qué impacto económico va a tener para la empresa?

EMPIEZA AQUÍ: Alto impacto + Bajo esfuerzo

Son las victorias rápidas.

Demuestran valor en semanas, no en meses. Generan confianza en los equipos de negocio y te dan la credibilidad necesaria para abordar proyectos más complejos.

PLANIFICA: Alto impacto + Alto esfuerzo

Son los proyectos transformadores.

Los que realmente cambian cómo funciona el negocio. Hay que planificarlos bien, conseguir apoyo directivo y recursos, y ejecutarlos en fases que generen valor incremental. No esperes a tenerlos perfectos para empezar.

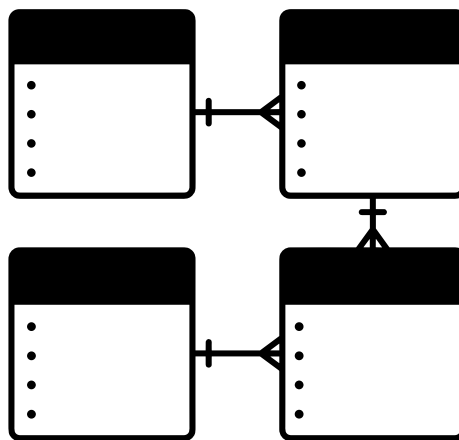
Para usar esta matriz **impacto vs. esfuerzo**, y que te ayude a resolver cada problema identificado, hazte dos preguntas: *¿Cuánto mueve la aguja del negocio si lo resuelvo?* *¿Cuánto tiempo y esfuerzo me costará resolverlo?*

Busca siempre victorias rápidas (proyectos de bajo esfuerzo y alto impacto) para ganarte la confianza de los directores en tus primeros meses.

Tú no estás para hacer análisis bonitos que acaban en un cajón, **estás para generar negocio.**

3. BASES DE DATOS RELACIONALES Y SQL — LA COLUMNA VERTEBRAL DE LOS DATOS EMPRESARIALES

Antes de entrar en la demo de Claude Code y que te muestre el tremendo potencial de lo que podemos lograr con la Inteligencia Artificial, necesitamos entender el ecosistema en el que viven los datos.



Mira, te lo digo claro: no importa si luego quieres especializarte en Machine Learning, en IA generativa o en hacer cuadros de mando visuales.

SQL y las bases de datos relacionales son tu punto de partida obligatorio. Son la forma en la que las empresas llevan décadas almacenando y organizando su información.

Querer hacer Data Science sin entender esto es como intentar escribir una novela sin saber el abecedario

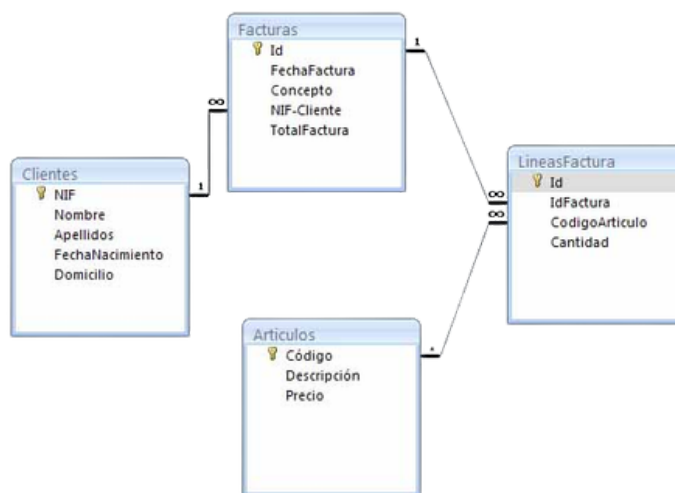
3.1 ¿Qué es exactamente una base de datos relacional?

Piénsalo como si fueran unas hojas de cálculo súper organizadas.

Una base de datos relacional es un **sistema que guarda la información en tablas**.

Cada tabla tiene sus filas (los registros) y sus columnas (los campos).

Pero lo que de verdad hace que esto sea "relacional" (y brillante) es que estas tablas no viven aisladas, sino que se conectan entre sí a través de campos comunes que llamamos "claves".



Para que lo veas con el **ejemplo** que usamos en *StyleData*: Imagina que tenemos dos tablas. Una de **ventas** (con la fecha, la tienda, el producto y el precio) y otra de **tiendas** (con el nombre de la tienda, la ciudad y el país).

En lugar de repetir el texto "*StyleData Madrid Centro, España*" miles de veces en cada línea de cada venta que se hace, simplemente guardamos un numerito, por ejemplo, `tienda_id = 1`. **Ese número apunta directamente a la fila correcta de la tabla de tiendas.**

```
ventas: id=5001 | tienda_id=1 | producto_id=3 | unidades=7 |  
precio=49.95
```

↓

```
tiendas: id=1 | nombre='StyleData Madrid Centro' | ciudad='Madrid'  
| mercado='ES'
```

Esto, que parece un detalle técnico menor, le da a las empresas **3 ventajas brutales**:

- **Cero duplicados:** Si mañana decidimos cambiar el nombre de una tienda, solo lo cambiamos una vez en la tabla principal, no en las miles de transacciones de ventas.
- **Consistencia total:** El sistema no te deja registrar una venta en una tienda que no existe. Las relaciones actúan como un candado de seguridad.
- **Máxima eficiencia:** Estas bases de datos están diseñadas para responder preguntas cruzando millones de registros a toda velocidad.

Ayer en la demo de StyleData trabajamos con una base de datos real compuesta por 10 tablas interconectadas. Fíjate en cómo lo teníamos estructurado:

TABLA	REGISTROS	QUÉ CONTIENE
ventas	22.272	Cada transacción: fecha, tienda, producto, unidades, precio y descuento.
tiendas	23	Las 23 tiendas físicas y online con su ciudad y mercado.
mercados	6	Los 6 mercados europeos: ES, FR, IT, DE, PT, GB.
productos	15	Los 15 productos con nombre, categoría, precio base y temporada.
categorías	6	Las 6 categorías: pantalones, camisetas, vestidos, chaquetas, accesorios...
inventario	540	Stock inicial, vendido, devuelto y final por tienda y temporada.
devoluciones	864	Devoluciones con fecha, tienda, producto, unidades y motivo.
pedidos_compra	24	Pedidos a proveedores con fechas acordadas, reales y cantidades.
proveedores	8	Los 8 proveedores con país, lead time, fiabilidad y mínimo de pedido.

3.2 SQL — El idioma universal (y el que te dará trabajo)

SQL (Structured Query Language) es el lenguaje que usamos para "hablar" con estas bases de datos.



Te cuento algo que sorprende a muchos: SQL lleva más de 50 años activo y, a día de hoy, sigue siendo la habilidad técnica más demandada en las ofertas de trabajo, muy por delante de Python o del Machine Learning.

¿Por qué? Porque el 90% de los datos empresariales del mundo viven ahí dentro. Si dominas SQL, te da igual cambiar de empresa o de sector; siempre sabrás cómo extraer la información.

TU "CHULETA" DE COMANDOS FUNDAMENTALES:

No necesitas saberte un manual de memoria. Con dominar estos comandos, ya puedes resolver el 80% de las preguntas que te haga el negocio:

1. **SELECT:** Para elegir qué columnas exactas quieres ver.
2. **FROM:** Para indicarle de qué tabla vienen esos datos.
3. **WHERE:** Para filtrar filas que cumplan una condición (ej: "Solo ventas de España").
4. **JOIN:** Para cruzar y unir dos tablas usando un campo común.
5. **GROUP BY:** Para agrupar los datos y sacar totales.
6. **ORDER BY:** Para ordenar los resultados de mayor a menor, alfabéticamente, etc.

COMANDO	EJEMPLO en StyleData
SELECT	¿Qué fecha, producto y unidades tienen las ventas?
FROM	De la tabla ventas
WHERE	Solo ventas de España en 2025
JOIN	Unir ventas con tiendas para saber el mercado
GROUP BY	Ventas totales agrupadas por mercado
ORDER BY	Los mercados de mayor a menor facturación

SQL EN ACCIÓN (LO QUE HICIMOS EN EL DIRECTO):

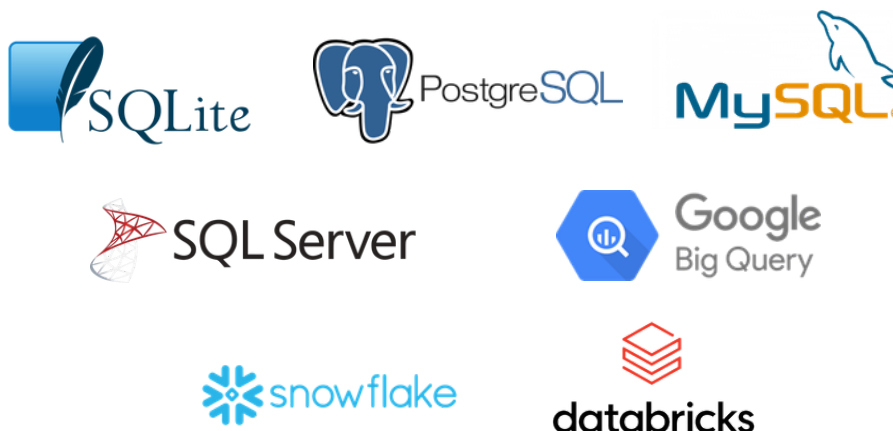
Aquí tienes las queries (consultas) reales que ejecutamos ayer con StyleData. Como ves, si lo lees despacio, es casi como leer en inglés básico:

```
-- SELECCIONAR TODOS LOS REGISTROS DE UNA TABLA
SELECT * FROM VENTAS;
-- FILTRAR VENTAS DE ESPAÑA EN 2025
SELECT FECHA, PRODUCTO_ID, UNIDADES, PRECIO_UNITARIO
FROM VENTAS
WHERE TIENDA_ID IN (SELECT ID FROM TIENDAS WHERE MERCADO_ID =
'ES')
AND FECHA >=
-- CALCULAR FACTURACIÓN TOTAL POR MERCADO
SELECT M.NOMBRE AS MERCADO,
SUM(V.UNIDADES * V.PRECIO_UNITARIO * (1 - V.DESCUENTO)) AS
FACTURACION
FROM VENTAS V
JOIN TIENDAS T ON V.TIENDA_ID = T.ID
JOIN MERCADOS M ON T.MERCADO_ID = M.ID
GROUP BY M.NOMBRE
ORDER BY FACTURACION DESC;
```


LOS "MOTORES" CON LOS QUE TE VAS A PELEAR AHÍ FUERA

SQL es un estándar, pero en el mundo real existen diferentes "marcas" o motores de bases de datos que lo usan. A lo largo de tu carrera te cruzarás con estos:

- **SQLite:** El que usamos ayer en la demo. Es genial porque no necesita servidor, todo vive en un archivo de tu ordenador. Perfecto para aprender y hacer prototipos.
- **PostgreSQL:** Súper potente y gratuito. Es el rey indiscutible en startups y empresas medianas modernas.
- **MySQL / MariaDB:** Lo verás muchísimo en e-commerce y desarrollo web (es lo que usan gigantes como Facebook, Twitter (X), o plataformas como WordPress).
- **SQL Server:** El ecosistema de *Microsoft*. Muy habitual si entras a trabajar en grandes corporaciones o bancos.
- **BigQuery:** La bestia de *Google* en la nube. Se usa para analizar terabytes de datos pagando solo por el uso, sin gestionar servidores.
- **Snowflake / Databricks:** Plataformas modernas donde se mezcla el SQL clásico con Machine Learning y datos en tiempo real.



Y te preguntarás... **¿por qué usamos SQLite en la demo de ayer?**

Básicamente por **agilidad**. Al no requerir servidor, podíamos hacerte la demostración sin instalaciones complejas. Pero quédate con esto: en una empresa real, los datos vivirán en un gran servidor de *PostgreSQL* o *BigQuery*, pero la lógica y el SQL que escribirás será exactamente el mismo que te enseñé ayer.

Aprender la base aquí te prepara para cualquier entorno.

4. DEMO CLAUDE CODE — UN ANALISTA DE DATOS EN LENGUAJE NATURAL

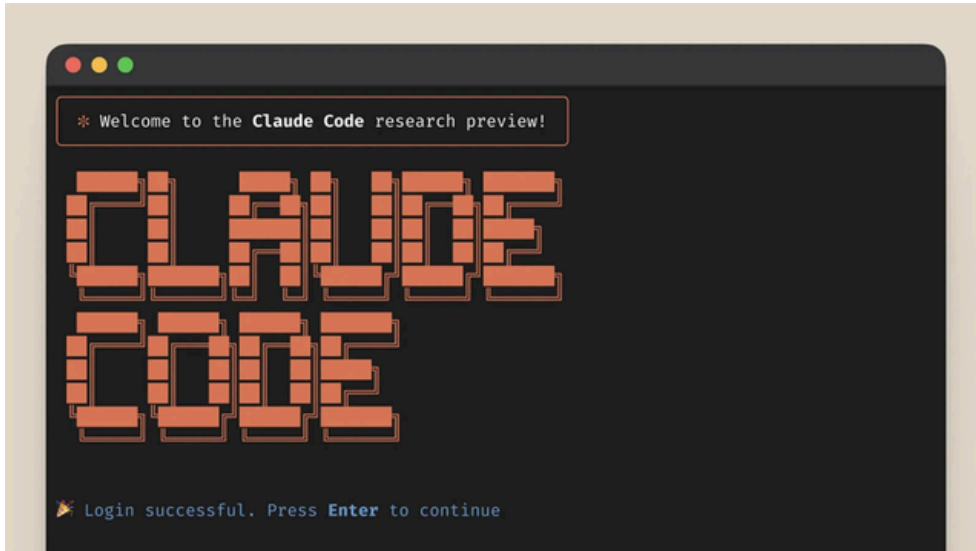
Ahora que ya tienes claro **qué es una base de datos y cómo funciona SQL**, podemos meternos de lleno en el sistema que montamos en la demo.

Mira, la idea central de este bloque es sencilla pero brutal: **conectar una Inteligencia Artificial directamente a tu base de datos** para que cualquier persona de la empresa (sepa o no de tecnología) pueda hacer **preguntas en lenguaje natural y obtener análisis automáticos**.

Imagina poder decirle a la IA: "*¿Cuál es el mercado con más ventas este año?*" y que ella sola entienda tu base de datos, escriba el SQL necesario, lo ejecute, extraiga los datos, te dibuje un gráfico y te escriba un resumen ejecutivo recomendándote qué hacer. [Eso es exactamente lo que montamos ayer.](#)

Y lo mejor de todo: **este sistema funciona igual si usas SQLite, PostgreSQL, BigQuery o Databricks**. Da igual cuál elijas.

4.1 ¿Qué es exactamente Claude Code?



Para la demo usamos [Claude Code \(de la empresa Anthropic\)](#). Y quiero dejar algo súper claro: esto no es un chatbot normal como ChatGPT al que le pides un texto y te lo escupe en la pantalla.

Claude Code es un **agente**. Y la diferencia es enorme. Un agente tiene permisos para actuar en tu ordenador.

Es capaz de:

1. Abrir, leer y crear archivos en tus carpetas.
2. Ejecutar comandos reales en tu terminal.
3. Correr scripts de Python, instalar librerías que le falten y lanzar procesos.
4. Generar visualizaciones visuales y guardar las imágenes.
5. Mantener todo un flujo de trabajo estructurado y organizado.

En lugar de que tú tengas que estar copiando y pegando código manualmente de una pestaña a otra, Claude actúa como un compañero de trabajo autónomo que ejecuta el proceso de principio a fin.

4.2 La arquitectura del sistema (Por qué funciona con cualquier base de datos)

Aquí es donde mucha gente se confunde. Creen que el secreto de este sistema es el código de *Python*. Y no. **El secreto es la arquitectura.**

Ojo a esto: la IA NUNCA habla directamente con tu base de datos a lo loco. Entre la IA y los datos hay una "**capa traductora**".

Fíjate en el **flujo** exacto que sigue el sistema cuando tú le haces una pregunta:

```
# La arquitectura del sistema:  
  
TU PREGUNTA EN ESPAÑOL  
↓  
Claude (la IA)  
↓  
Lee context/schema.md ← Sabe qué tablas y columnas existen  
↓  
Genera el SQL correcto  
↓  
run_query.py ← Ejecuta el SQL en tu base de datos  
↓  
Resultados en CSV  
↓  
Genera gráfico + resumen ejecutivo
```

¿Por qué lo montamos así?

Porque si mañana entras a trabajar a una empresa que usa *BigQuery* en lugar de *SQLite*, solo tienes que **cambiar el script de conexión** (`run_query.py`). El resto de la inteligencia, los gráficos y el flujo, siguen funcionando intactos.

4.3 Los dos scripts clave del sistema

Para que esta arquitectura funcione, vimos que necesitábamos **dos pequeños scripts en Python**. Vamos a verlos en detalle:

Script 1: `describe_schema.py` (El explorador)

Este script entra a tu base de datos, mira qué hay dentro y genera automáticamente un archivo de texto llamado `context/schema.md`.

¿Para qué? Para darle a la IA un **mapa del terreno**. Sin esto, la IA iría a ciegas e inventaría nombres de tablas que harían fallar el SQL. Con este mapa que le damos antes de empezar, acierta a la primera.

```
# Ejecución del script explorador
python scripts/describe_schema.py

# Resultado: genera context/schema.md con toda la estructura
# Ejemplo del contenido generado:
#
# ## `ventas` — 22.272 filas
# | Columna | Tipo | Ejemplos |
# |-----|-----|-----|
# | fecha | DATE | 2024-04-01, 2024-04-08 |
# | tienda_id | INTEGER | 1, 2, 3, 4 |
# | unidades | INTEGER | 7, 8, 6, 5 |
# | precio_unit. | REAL | 49.95, 33.47, 39.46 |
```

Script 2: `run_query.py` (El ejecutor)

Este es el "brazo ejecutor". Recibe el SQL que ha pensado la IA, lo lanza contra la base de datos y te devuelve los resultados (o los guarda en un CSV si se lo pides para hacer un gráfico).

```
# Uso básico — muestra resultados en pantalla
python scripts/run_query.py "SELECT * FROM ventas LIMIT 10"

# Guardando resultados en CSV para visualizaciones
python scripts/run_query.py "SELECT mercado, SUM(unidades) as total
FROM ventas JOIN tiendas ON ventas.tienda_id = tiendas.id
GROUP BY mercado ORDER BY total DESC" --save queries/ventas_por_mercado.csv

# La IA llama a este script automáticamente.
# Nunca conecta directamente a la base de datos.
```

4.4 El cerebro de la operación: CLAUDE.md

Si los scripts son los brazos, el archivo `describe_schema.py` es el cerebro absoluto. Este es tu manual de operaciones.

Acuérdate de lo que vimos: no es un archivo de código incomprensible, son instrucciones en texto plano donde tú le dictas a Claude las 4 reglas del juego obligatorias que tiene que seguir siempre que le pidas un análisis:

- **PASO 1:** Redactar la query SQL y guardarla como `.sql` en la carpeta `queries/`.
- **PASO 2:** Ejecutar la query usando `run_query.py` y guardar los resultados como `.csv`.
- **PASO 3:** Leer ese CSV, generar un gráfico con la librería `matplotlib` y guardarlo como `.png` en la carpeta `visualizations/`.
- **PASO 4:** Escribir el resumen ejecutivo en `insights/` (con la pregunta inicial, la metodología, los hallazgos clave y una recomendación de negocio).

IMPORTANTE:

Dentro de este `CLAUDE.md` también le obligamos a que, antes de hacer nada, se lea siempre el `schema.md` (para conocer las tablas) y el `business_rules.md` (para conocer el contexto de la empresa).

Tú pones las reglas, **él hace el trabajo pesado.**

4.5 Cómo generar estos scripts para tu propia empresa (Sin picar código)

Y aquí viene la lección más importante de toda la sesión: **tú no necesitas escribir estos scripts desde cero.**

El valor del nuevo Data Scientist no está en picar código de memoria, está en saber **qué pedirle a la IA, entender lo que genera y adaptarlo.** Ayer te mencioné los prompts exactos que usamos para generar los scripts del sistema. Úsalos tal cual en tu trabajo. Aquí los tienes para copiar y pegar.

👉 Prompt para crear el script explorador (describe_schema.py) :

"Necesito un script Python llamado describe_schema.py que se conecte a [TIPO DE BASE DE DATOS: PostgreSQL / BigQuery / Databricks / etc.]. Credenciales / configuración: Host: [tu host] | Base de datos: [nombre] | Usuario: [usuario]

El script debe:

1. Conectarse a la base de datos.
2. Listar todas las tablas disponibles.
3. Para cada tabla: listar columnas, tipos de datos y valores nulos.
4. Contar las filas exactas de cada tabla.
5. Mostrar 3-4 ejemplos de valores reales por columna.
6. Guardar todo en context/schema.md en formato tabla markdown."

👉 Prompt para crear el script ejecutor (describe_schema.py) :

"Necesito un script Python llamado run_query.py que:

1. Reciba una query SQL como primer argumento de línea de comandos.
2. Se conecte a [TIPO DE BD] con las mismas credenciales.
3. Ejecute la query y muestre los resultados en formato tabla en la terminal.
4. Si se pasa el argumento --save ruta.csv, guarde los resultados en ese CSV.
5. Si la query falla, muestre el error de forma clara.
6. Devuelva código de salida 0 si va bien, 1 si hay error."

4.6 El resumen ejecutivo — Lo que nos dijeron los datos de StyleData

Con el sistema funcionando, lanzamos los primeros análisis sobre los datos reales de StyleData.

Esto es el resumen ejecutivo que obtuvimos, exactamente como lo presentaríamos al equipo directivo:

Análisis 1 — Pareto de mercados y categorías

Hallazgo: España y Francia generan el 82% de la facturación total siendo solo 2 de los 6 mercados. Dentro de España, 3 tipos de chaquetas concentran el 42% del revenue del mercado.

Implicación para el negocio: El Pareto de StyleData es muy concentrado. Cualquier problema en España o Francia tiene un impacto enorme en el resultado global. Antes de diversificar, hay que asegurarse de que los mercados clave funcionan perfectamente.

Análisis 2 — Sobrestock por mercado

Hallazgo: Solo España y Alemania tienen tasas de sobrestock por debajo del umbral preocupante (20%). El resto — Portugal, UK, Francia — llevan dos temporadas consecutivas en zona preocupante sin corrección.

Implicación para el negocio: España genera el 56% de las pérdidas potenciales por sobrestock pese a tener el porcentaje más bajo. La razón: opera a una escala tan grande que cualquier ineficiencia vale mucho dinero en términos absolutos.

Análisis 3 — Estacionalidad y patrones por categoría

Hallazgo: Abrigo y Chaqueta Acolchada repiten exactamente el mismo patrón estacional año tras año, con un coeficiente de variación muy bajo. Son las categorías más predecibles de toda la base de datos.

Implicación para el negocio: Alta predecibilidad significa mayor probabilidad de que un modelo de forecast acierte desde el primer mes, lo que hace estas categorías los candidatos perfectos para una POC. Si el modelo funciona aquí, podemos escalar con confianza al resto.

Pero...¿Qué le decimos al negocio?

Tenemos un problema de planificación claro y sistemático, no puntual. Los datos nos dicen exactamente por dónde empezar: abrigos y chaquetas acolchadas en España. Además, tenemos un segundo problema de acceso a la información: cientos de contratos que nadie consulta porque buscar en ellos es ineficiente.

De este análisis nacen directamente los dos proyectos que vamos a proponer. No son ideas arbitrarias — están respaldados por los datos que acabamos de ver.

Eso es lo que diferencia una propuesta de valor real de una simple intuición.

5. LA METODOLOGÍA — DEL PROBLEMA A LA SOLUCIÓN

Antes de meternos de cabeza a ver los proyectos concretos que montamos para StyleData, necesitamos hablar de algo fundamental: el marco de trabajo.

Mira, me da igual si vas a montar un modelo predictivo súper complejo o un análisis sencillo de ventas; todo proyecto de Data Science en la vida real sigue siempre el mismo ciclo de 5 fases.

Grábatelas a fuego, porque **conocerlas te da la estructura mental para no perderte** y, sobre todo, para saber **explicarle tu trabajo a los directores de negocio**.

Las 5 fases innegociables de todo proyecto:

- **Definir el problema:** ¿Qué decisión de negocio queremos mejorar exactamente? ¿Qué métrica se va a ver afectada? Si empiezas a picar código **sin tener una pregunta clara** y un criterio de éxito medible, **no hay forma de saber si el proyecto ha servido para algo**. Te sorprendería saber que esta es la fase más importante y la que casi todo el mundo se salta.
- **Recolectar y preparar los datos:** Toca mancharse las manos. Hay que obtener los datos, limpiarlos, unirlos y transformarlos. Te aviso desde ya: **los datos de una empresa nunca llegan listos**. Siempre hay valores nulos, formatos raros o duplicados. Esta fase se va a comer hasta el 80% de tu tiempo en el proyecto, y es normal.

- **Capturar patrones (entrenar el modelo):** Aquí es donde el modelo aprende del pasado. Le aplicamos el algoritmo adecuado para que encuentre relaciones ocultas en el histórico de datos (por ejemplo, qué combinación de factores hace que una chaqueta se venda más).
- **Aplicar los patrones (inferencia):** Una vez que el modelo ha aprendido, lo usamos para hacer predicciones sobre datos nuevos que nunca ha visto. El modelo no "adivina", sino que aplica la lógica que aprendió del pasado a las situaciones del futuro. En una empresa, esto ocurre de forma automatizada todos los días.
- **Medir, evaluar y mejorar:** Comparamos lo que dijo el modelo con lo que pasó en la realidad. Un modelo nunca se da por "terminado". Hay que vigilarlo continuamente porque los patrones de los clientes cambian, y cuando eso pasa, toca reentrenarlo.

¿Y DÓNDE ENTRA LA IA EN TODO ESTO?

La Inteligencia Artificial Generativa **te puede ayudar en todas las fases:** puede explorar los datos por ti en la fase 2, sugerirte qué algoritmo usar en la fase 3, generarte el código para evaluar en la fase 5 y hasta redactarte el informe final.

Pero ojo: la decisión de qué hacer, cuándo hacerlo y qué significa ese dato para la empresa... **esa responsabilidad sigue siendo 100% tuya.**

6. PROYECTO 1 — DEJA DE ADIVINAR CUÁNTO PRODUCIR

Ayer, revisando los datos de StyleData, vimos claro que el problema de que sobre ropa en las tiendas no es algo puntual, es un error sistemático.

Cada temporada, el equipo de compras produce de más en unos países y de menos en otros simplemente porque no tienen herramientas para anticiparse

6.1 El problema en detalle

LO QUE PASA HOY	LO QUE QUEREMOS
• Los pedidos a proveedores se hacen por intuición y experiencia del comprador • Cada temporada sobra stock que acaba en rebajas con pérdida de margen • A veces falta producto y se pierden ventas directamente • No hay visibilidad de qué mercados necesitan qué producto	• Predecir cuántas unidades se venderán por tienda, mercado y categoría • Con al menos 2-3 meses de antelación para poder hacer el pedido • Reducir el sobrestock sin aumentar la rotura de stock • Dar al equipo de compras una herramienta que respalde sus decisiones con datos

Para que entiendas el reto al que nos enfrentábamos, fíjate en el punto de partida y a dónde queríamos llegar:

- **El drama actual:** El equipo hace los pedidos a los proveedores basándose puramente en su intuición. El resultado es que sobra stock (que se malvende en rebajas perdiendo margen) y a veces falta producto estrella (perdiendo ventas directas).
- **Nuestro objetivo:** Queremos predecir exactamente cuántas unidades se van a vender por tienda, mercado y categoría, **con 2 o 3 meses de antelación** para que el equipo tenga margen para encargar la producción.

6.2 La solución: Forecast de demanda — Paso a paso

FASE 1 — DEFINIR EL PROBLEMA (Y NO PASARNOS DE LISTOS)

Antes de tocar nada, hay que tener claro el alcance. En StyleData los proveedores tardan una media de 8 semanas en entregar la ropa (lead time). Si nuestro modelo hace predicciones a 3 semanas vista, no le sirve de nada al equipo de compras porque el pedido ya estaría cerrado.

Tampoco podemos ponernos a predecir prenda por prenda en cada tienda individual porque nos faltaría histórico. Hay que buscar el equilibrio. Decidimos empezar con una Prueba de Concepto (POC) solo para las categorías de Abrigos y Chaquetas acolchadas en España, ya que es justo como el equipo organiza sus pedidos hoy.

Acordamos estos criterios de éxito con los directores para esta POC:

1. Un margen de error (MAPE) inferior al 15%.
2. Un Bias cercano a cero (el modelo no puede tirar sistemáticamente por lo alto ni por lo bajo).
3. Que la respuesta sea fácil de entender para el equipo de compras, sin tecnicismos.

FASE 2 — PREPARAR LOS DATOS (DONDE OCURRE LA MAGIA)

Aquí es donde sudamos la camiseta. Necesitamos el histórico de ventas desde 2024, cruzarlo con el calendario de festivos, el Black Friday y el tráfico de la web.

Tuvimos que hacer 4 cosas clave:

1. **Agrupar por semanas:** La base de datos guarda cada ticket de compra diario, pero nuestro modelo necesita ver las ventas totales agrupadas por semana.
2. **Cuidado con las tiendas nuevas:** Si una tienda abrió hace dos meses, el modelo no tiene datos suficientes para entender su ciclo anual. Para esta primera prueba, la sacamos de la ecuación y usamos solo tiendas consolidadas.
3. **Etiquetar las rebajas:** Las rebajas de enero rompen cualquier gráfica de ventas. Si no le decimos al modelo "oye, este pico de ventas es porque la ropa estaba a mitad de precio", pensará que es el ritmo normal y pedirá demasiada ropa para febrero.
4. **No hacer trampas al solitario (Train vs. Validation):** Usamos los datos de abril 2024 a junio 2025 para "enseñar" al modelo, y luego le pedimos que intentara predecir lo que pasó de julio a octubre de 2025.

Comparamos su predicción con la realidad. Nunca, jamás, mezcles datos del futuro en la fase de entrenamiento.

FASE 3 — ENTRENAR EL MODELO (BUSCANDO PATRONES)

¿Qué algoritmo usamos?

Empezamos siempre por lo más sencillo (ARIMA o Regresión Lineal).

Pero ¿Por qué? **Porque son fáciles de explicar.**

Un modelo simple que el equipo de compras entiende, genera muchísima más confianza que meterle de golpe un algoritmo complejo como *XGBoost* que nadie sabe cómo ha llegado a esa conclusión.

FASE 4 — LA INFERENCIA (DARLE EL DATO AL NEGOCIO)

Ya tenemos el modelo. Ahora, cada vez que Compras tiene que hacer un pedido, el modelo escupe la predicción. Pero no les damos un gráfico lleno de fórmulas, les damos una tabla súper masticada:

El modelo no despide al comprador, le da un apoyo brutal basado en datos objetivos para que él tome la decisión final.

Ejemplo de output del modelo para el equipo de compras de España:

Mercado: España | Categoría: Chaquetas | Semana: 38/2026 | Unidades previstas: 1.340 | Rango: 1.180 – 1.500

Mercado: España | Categoría: Chaquetas | Semana: 39/2026 | Unidades previstas: 1.520 | Rango: 1.310 – 1.730

Mercado: España | Categoría: Abrigos | Semana: 38/2026 | Unidades previstas: 890 | Rango: 780 – 1.000

FASE 5 — MEDIR, EVALUAR Y MEJORAR

En la vida real, los patrones cambian. Lo que funcionó en 2025 puede fallar en 2026. Por eso vigilamos de cerca estas métricas:

1. **MAPE:** El error porcentual. Si es del 12%, significa que nos equivocamos en 12 chaquetas de cada 100.
2. **Bias (Sesgo):** Como comerse el stock cuesta muchísimo dinero, configuramos el modelo con un sesgo ligeramente negativo. Preferimos predecir un pelín a la baja (y tener que reponer) antes que pasarnos y comernos miles de abrigos en el almacén.
3. **Tasa de adopción:** Esta es la métrica reina. Si tenemos un modelo buenísimo, pero el equipo de compras no lo mira para hacer sus pedidos, nuestro impacto es cero.

6.3 Plan de Rollout (Cómo escalar sin estrellarnos)

Las cosas en la empresa se hacen paso a paso:

1. **POC (Prueba de concepto):** Empezamos solo con abrigos en España porque es el producto más estable. Buscamos una **victoria rápida** para que los directivos confíen en nosotros.
2. **Piloto:** Los compradores empiezan a usar nuestras tablas junto a su intuición y nos dan **feedback**.
3. **Escalado:** Cuando confían en el sistema a ciegas, lo **desplegamos** para los 6 países y para todas las prendas de la marca

7. PROYECTO 2 — DE CIENTOS DE CONTRATOS A UNA RESPUESTA EN SEGUNDOS

El segundo proyecto que presentamos para *StyleData* no va de predecir el futuro, **va de ordenar el caos del presente.**

Acuérdate del drama que nos planteó el equipo de compras: tienen contratos con decenas de proveedores. Cada contrato es un PDF infumable de entre 15 y 60 páginas lleno de cláusulas, penalizaciones, mínimos de pedido y plazos de entrega.

7.1 El problema real (Más allá de perder el tiempo)

¿Qué pasa hoy cuando alguien del equipo necesita consultar algo urgente?

1. Se vuelve loco buscando el contrato en carpetas de red desorganizadas (a veces abriendo una versión que ni siquiera es la vigente).
2. Abre el PDF y le da al clásico "Ctrl+F" para buscar una palabra. Y aquí viene el problema: si tú buscas la palabra "tiempo de envío" pero el abogado que redactó el contrato escribió "plazo de entrega", el buscador no encuentra nada.
3. Pierden entre 30 minutos y varias horas por consulta.

Pero ojo, el coste real no es el tiempo perdido. **El riesgo brutal para el negocio** es tomar una decisión de compra importante (o comerte una penalización) basándote en una cláusula desactualizada o que no has sabido encontrar.

7.2 La solución: RAG (Retrieval-Augmented Generation)

Para arreglar esto, diseñamos un sistema **RAG**. Básicamente, es una técnica que une lo mejor de dos mundos: un buscador súper potente basado en el significado de las palabras, y un modelo de lenguaje (como ChatGPT) que redacta la respuesta.

La diferencia clave con hacer "Ctrl+F" es que el RAG busca por significado semántico, no por palabras exactas. Entiende que "plazo de entrega" y "días hasta recepción" son la misma cosa. Y lo más importante: no inventa nada. Solo responde usando el texto de tus propios PDFs.

Ayer vimos cómo se monta esto aplicando nuestras **5 fases metodológicas**:

FASE 1 — DEFINIR EL PROBLEMA Y LOS USUARIOS

No es lo mismo pedirle a la IA un dato concreto ("*¿Cuál es el plazo de entrega de TextilPro?*") que pedirle una comparativa compleja ("*¿Qué proveedor tiene mejores condiciones de cancelación?*"). Para nuestra Prueba de Concepto (POC), nos centramos solo en preguntas directas y concretas. Buscamos victorias rápidas para que el equipo de compras (que no es técnico) empiece a confiar en la herramienta.

FASE 2 — PREPARAR E INDEXAR LOS DOCUMENTOS (LA FASE CRÍTICA)

Los PDFs son un dolor de cabeza técnico. Para que la IA los entienda, hicimos esto:

- Extraemos el texto limpio conservando la estructura (títulos, tablas, etc.).
- Lo cortamos en trocitos manejables que llamamos **chunks** (normalmente de entre 300 y 500 palabras, respetando los límites de cada cláusula).
- Convertimos cada chunk en una serie de números matemáticos (los famosos **embeddings**) y los guardamos en una **Base de Datos Vectorial**.
- Truco PRO: A cada trozo le añadimos "metadatos" (proveedor, fecha, número de contrato). Sin esto, la IA luego no podría decirte "Según el contrato de 2024 de TextilPro...".

FASE 3 — RECUPERAR LOS FRAGMENTOS (LA BÚSQUEDA)

Cuando el usuario lanza su pregunta, el sistema la convierte también en un vector matemático.

Luego, busca en la base de datos y extrae los fragmentos (normalmente entre 3 y 8 chunks) que matemáticamente más se parecen al significado de la pregunta.

FASE 4 — GENERAR LA RESPUESTA (CERO ALUCINACIONES)

Aquí entra el modelo de lenguaje. Le pasamos la pregunta del usuario, le adjuntamos los fragmentos que hemos encontrado y le damos una orden estrictamente militar en el prompt:

"Responde ÚNICAMENTE usando la información de estos fragmentos. Si la respuesta no está ahí, di que no lo sabes. NO INVENTES NADA".

Esta regla es innegociable en entornos legales y de compras.

FASE 5 — EVALUAR Y MEJORAR (¿CÓMO MEDIMOS ESTO?)

Evaluar un RAG no es como evaluar un modelo predictivo numérico (aquí **no hay un "MAPE"**). La evaluación es mixta:

- **Automatizada:** Le lanzamos preguntas trampa de las que ya sabemos la respuesta y medimos la tasa de acierto, la tasa de alucinaciones (inventos) y la tasa de "No sé" (que a veces es la respuesta correcta si el dato no está en el PDF).
- **Humana:** En producción, le ponemos al equipo de compras un botoncito de 👍 (pulgar arriba) y 👎 (pulgar abajo) en cada respuesta. Cada semana revisamos los pulgares abajo para afinar el sistema.

7.4 Plan de Rollout (Por qué lo lanzamos junto al Forecast)

Las cosas se escalan con cabeza. Fíjate cómo lo planteamos:

FASE	QUÉ HACEMOS EXACTAMENTE
Fase 1 (POC)	Empezamos solo con los contratos de los mercados donde hicimos el Forecast. Acotado y controlado.
Fase 2 (Piloto)	El equipo de compras lo empieza a usar en su día a día. Validamos la precisión de las respuestas con ellos.
Fase 3 (Escalado)	Lo abrimos a todos los contratos de la empresa y le damos acceso a Comercial y al departamento Jurídico.

¿Por qué lanzamos Forecast y RAG a la vez? Porque son la pareja perfecta.

- El modelo de Forecast le dice a Compras cuánta ropa tiene que pedir.
- El sistema RAG le dice bajo qué condiciones legales la tiene que pedir.

Con estas dos herramientas, acabas de blindar el departamento entero.

8. LAS 4 HABILIDADES QUE HACEN CARRERA

Para cerrar los apuntes (y la sesión de ayer), quiero dejarte una reflexión vital.

La diferencia entre un Data Scientist que sobrevive "picando código" y uno que escala en su carrera hasta los sueldos más altos no está en saberse de memoria una librería nueva de Python.

Está en **4 habilidades transversales** que rara vez te enseñan en la universidad, pero por las que las empresas se pelean:

1. Pensamiento crítico (Haz las preguntas correctas)

La IA escribe código y calcula métricas, pero no sabe si el problema que estás resolviendo tiene sentido. No asumas nunca que el dataset está limpio y no confundas correlación con causalidad.

Ejemplo real: Tu modelo predictivo acierta con un 99% de precisión. En vez de ir a celebrarlo, tu pensamiento crítico te hace frenar y preguntar: "Espera, ¿no será que el modelo está usando datos del futuro para predecir el presente (lo que llamamos Data Leakage)?". Tu criterio salva al negocio de un desastre en producción.

2. Storytelling con datos (Tu análisis no habla solo)

Un gráfico espectacular que el negocio no entiende, es un gráfico inútil. Tienes que saber contar la historia: da el contexto, muestra el hallazgo, explica la implicación y haz una recomendación directa.

✗ **Perfil Junior sin storytelling:** "La tasa de sobrestock en Portugal es del 31.4% con un desvío estándar del 8.2%."

✓ **Perfil Senior con storytelling:** "Portugal lleva dos temporadas en zona crítica de sobrestock. Esto nos está costando X€ de margen perdido en rebajas. Si no ajustamos las entregas de octubre, el patrón se va a repetir."

3. Impacto de negocio (El idioma del dinero)

Aplica siempre la matriz de impacto vs esfuerzo. Cada hora que pasas intentando subir un 1% la precisión de un modelo que a nadie le importa, es una hora perdida. Y cuando presentes tus proyectos a los directivos, no hables de tecnología, habla de euros.

Aplica siempre la matriz de impacto vs esfuerzo. Cada hora que pasas intentando subir un 1% la precisión de un modelo que a nadie le importa, es una hora perdida. Y cuando presentes tus proyectos a los directivos, no hables de tecnología, habla de euros.

4. Adaptabilidad (Las herramientas cambian, tú no)

Hace 5 años nadie hablaba de LLMs. Hace 2 años nadie hablaba de agentes autónomos. La tecnología va a cambiar de forma bestial. Pero **si tú dominas los fundamentos de verdad** (Estadística, SQL, pensamiento crítico, comunicación y foco en el negocio), **aprenderás cualquier herramienta nueva en dos semanas**. Quien solo sabe "apretar los botones" de un programa concreto, se queda obsoleto rápido.

COMO TE DIJE PARA DESPEDIR LA CLASE:

*"El Data Scientist del futuro no es el que más herramientas técnicas domina. **Es el que sabe preguntar, interpretar y comunicar.**"*

9. Y AHORA, ¿CUÁL ES EL SIGUIENTE PASO?

Durante las jornadas hemos visto solo la punta del iceberg de lo que puedes llegar a conseguir dominando los datos y apoyándote en la Inteligencia Artificial.

Hemos visto cómo predecir el futuro, cómo hacer recomendadores y cómo montar un sistema RAG corporativo. Aunque una cosa es asistir a dos clases, entender el concepto y adquirir nociones de datos... y otra muy distinta es **convertir esto en tu profesión y empezar a liderar proyectos en el mundo real.**

Para eso exactamente hemos diseñado el [Máster de Data Science con IA de BIG school.](#)



Da igual si partes completamente desde cero y quieres reinventarte, o si ya eres un profesional IT que busca actualizarse.

El Máster está preparado para llevarte de 0 a 100.



**PROFESIONALES
DE OTRO SECTOR**



**PROFESIONALES
DE LA TECNOLOGÍA**



PARTES DE CERO

Vamos al grano. Esto es lo que te llevas si decides dar el paso y entrar en el Máster:

+135
MÓDULOS

+400
HORAS

+500
LECCIONES

El Temario (6 meses, 100% práctico y directo a producción)

Aquí no venimos a hacer código que se queda en tu ordenador. Venimos a crear soluciones de negocio. Durante el Máster vas a dominar

- **Fundamentos y Data Science clásico:** Data Thinking, Series temporales, Machine Learning y Deep Learning.
- **Inteligencia Artificial Generativa (La artillería pesada):** Nos meteremos de lleno en Large Language Models (LLMs), Prompting avanzado, Fine-Tuning, Embeddings, Sistemas RAG, Text-to-speech y Visión por Computador.
- **Agentes y Multiagentes:** Vas a aprender a crear no solo IAs que responden, sino agentes autónomos que ejecutan tareas.
- **Despliegue y Producción:** Aprenderás a pasar "De Notebooks a Scripts". Veremos ML en Producción, Airflow, MLFlow, Serving y cómo montar tu repositorio profesional en GitHub.

El Ecosistema (Acceso Vitalicio. Esto lo cambia todo)

La tecnología cambia rapidísimo. Por eso, en BIG school pagas una vez y te preparas para siempre. Tienes acceso vitalicio a las clases grabadas y a todas las actualizaciones futuras del temario.

Nunca te vas a quedar obsoleto. Además, no estarás solo. Tienes tutorías semanales en directo para siempre y soporte individualizado (1 a 1) durante 12 meses.

Y **acceso a la Comunidad privada de Discord 24/7**, de esta y otras ediciones.

Los Profesores (Expertos en la trinchera)

No vas a aprender solo teoría infumable. Vas a aprender de profesionales que auditan y resuelven problemas en empresas reales cada día.

Me tendrás a mí (Gerard Sánchez), **junto a cracks de la industria** como Raúl Carbajosa, Marc Herrera, Nuria Cantero, Daniel Arias, Gustavo Cueva, Davide Redondi y Mario Vázquez (entre otros).

Doble titulación que acredita tu talento en los entornos más competitivos

Para que las empresas no tengan dudas de tu nivel, te llevas una Doble Titulación al terminar el Máster:

- Título de la Escuela de Negocios BIG school.
- Título Universitario de la Universidad Europea Miguel de Cervantes (UEMC) con 30 créditos ECTS, lo que te abre la puerta a hacer prácticas en empresas si lo deseas.



Los BONUS Exclusivos de esta edición

Por si fuera poco, al formalizar tu matrícula te llevas de regalo:

- **4 Certificados Independientes:** A medida que avanzas en el Máster, vas desbloqueando tu perfil técnico nivel a nivel con 4 certificaciones: Data Analyst, Data Engineer, Data Scientist y AI Engineer. Ideal para destacar en el mercado laboral mucho antes de terminar la formación.
- **Masterclasses Exclusivas:** Veremos casos de uso impresionantes, como el uso de Datos Satelitales y cómo se aplica el Machine Learning en gigantes como AirBnB.
- **Workshops de Alto Rendimiento:** Entrenamientos prácticos creados por Data Scientists en activo. Con un objetivo es claro: ayudarte a pensar, analizar y decidir como un profesional real del sector.
- **Y la entrada a la BIG Fiesta:** Nuestro evento presencial para desvirtualizarnos, hacer networking y pasarlo en grande

"Gerard, ¿y si entro y veo que esto no es para mí?"

Como buen perfil de datos, entiendo que quieras minimizar el riesgo al máximo. Por eso tienes **15 días de garantía total de devolución**.

Entras, accedes al contenido, te metes a Discord, asistes a los primeros directos y, si en esas dos semanas ves que no era lo que esperabas, pides la devolución y te reintegramos el 100% de tu dinero.

Sin letras pequeñas y sin hacerte preguntas incómodas.

[Mira todos los detalles en la página del Master de Data Science con IA](#)

Si has visto claro que dominar los datos y la IA es la pieza que le falta a tu carrera para dar ese salto definitivo, **me encantará darte la bienvenida personalmente en el primer directo del Máster.**

¡Nos vemos dentro!

Gerard Sánchez

Director del Master de Data Science con IA

BIG school

MASTER DE DATA SCIENCE CON IA

**CONVIÉRTETE EN
EL PERFIL QUE
BUSCAN LAS
EMPRESAS**

MATRICÚLATE AHORA

BIG school